

mcp-ratchet — Tool Verification Cases & Results

The executed operational verification: one or more requirements-based cases per Tool Operational Requirement, run on the qualified environment, with recorded results and the code changes the verification drove.

MR-TQ-003 §2 assigns the bulk of a TQL-5 effort to *tool operational verification — requirements-based testing in the operational environment*. This document is that verification, executed. Each of TOR-1 through TOR-11 (MR-TQ-002 §3) has at least one verification case with a defined input, an expected result, and a recorded result. Where the current source did not meet a TOR when MR-TQ-002 was written, the code change that closed it is identified by commit.

§1 Method and what counts as evidence

Verification is requirements-based: a case exists to demonstrate one TOR, not to exercise code for its own sake. Cases were executed by running the mcp-ratchet automated test suite (`python -m pytest`) against the repository at commit 907c8a9 on the environment recorded in §2. Three forms of evidence are cited:

- Automated test cases** — named tests under `tests/` , each asserting a specific TOR behaviour. These are the primary evidence for TOR-1 through TOR-7 and TOR-10.
- Repository commits** — for the three TORs whose verification required a source change (TOR-8, TOR-9, TOR-11), the commit that made the change, so the qualified behaviour is tied to an identified configuration item.
- Real generated artifacts** — for the TOR-8 and TOR-11 *report markers*, the check is run against an actually generated `reports/dashboard.html` and `reports/drift-summary.json` , per MR-TQ-003 §9 ("on a real generated report, not just the template source").

NOT CLAIMED HERE

This verification was performed by the tool's author. DO-330 expects verification independent of development (MR-TQ-003 §7). That limitation is unresolved and is carried to MR-TQ-005 as a condition on any real use of this material.

§2 Operational environment, as run

ITEM	VALUE AT VERIFICATION	AGAINST MR-TQ-002 §2
Host OS	Windows 10, 64-bit (build 10.0.19045)	matches
Python	CPython 3.13.5	matches (qualified point is 3.13)
MCP SDK	mcp 2.1.1	satisfies <code>mcp >= 2.0.0</code> ; see §6 — the <code>Tool</code> field set (TOR-1) was re-checked and is unchanged from 2.0.0
Fingerprint schema	FINGERPRINT_SCHEMA_VERSION = 1	matches
mcp-ratchet	commit 907c8a9 on main	candidate qualified commit (CM action in §7)
Suite result	194 passed, 0 failed, 0 skipped	—

§3 Verification results — per TOR

Each row: the requirement in brief, the verification case(s) demonstrating it, the recorded result, and the verdict. Full case source is in the named test file.

TOR	Requirement (Brief)	Verification Case(s)	Recorded Result	Verdict
TOR-1	Baseline records every observable <code>Tool</code> field; absent is recorded as absent, not null.	<code>test_fingerprint.py</code> : <code>records_per_tool_hashes</code> , <code>none_valued_optional_fields_do_not_leak_into_hash</code> , <code>fingerprint_round_trips_through_dict</code>	Canonical form carries name / title / description / input_schema / output_schema / annotations / icons / meta. A <code>None</code> -valued optional field is dropped, not hashed as null. Fingerprint survives a <code>to_dict</code> / <code>from_dict</code> round trip unchanged.	PASS
TOR-2	Match iff canonical forms are byte-identical: key order and formatting don't count, array order and any string change do.	<code>test_fingerprint.py</code> : <code>identical_tool_hashes_identically_across_calls</code> , <code>dict_key_order_does_not_affect_hash</code> , <code>whitespace_only_change_still_changes_hash</code> , <code>schema_change_changes_hash</code>	Repeat fingerprints of one tool hash identically. Reordering object keys does not change the hash. A whitespace-only description edit does change it. An <code>input_schema</code> edit does change it.	PASS
TOR-3	A tool on the current surface but not the baseline → drift, identified as added.	<code>test_drift.py</code> : <code>new_tool_added_between_two_runs_is_flagged_as_tool_added</code> (live subprocess: baseline the toy server, add a tool to its source, reconnect)	Exactly one drift event of any kind (<code>assert len(events) == 1</code>), of type <code>tool_added</code> , <code>tool_name</code> = the added tool. Nothing else flagged.	PASS
TOR-4	A tool in the baseline but not on the current surface → drift, identified as removed.	<code>test_drift_classification.py</code> : <code>removed_tool_is_flagged_as_tool_removed</code>	One <code>tool_removed</code> event for the dropped tool; <code>current_hash</code> is <code>None</code> ; no spurious events for the retained tool.	PASS
TOR-5	A tool in both whose canonical forms differ → drift, naming the tool and each changed field with baseline and current values.	<code>test_drift.py</code> : <code>description_change_between_two_runs_is_flagged_as_drift</code> (live). <code>test_drift_classification.py</code> : <code>input_schema_change_is_flagged_as_schema_changed</code> , <code>annotations_change_is_flagged_as_annotations_changed</code> , <code>multiple_fields_changing_at_once_each_reported</code>	Field-level classification correct for description, <code>input_schema</code> , and <code>annotations</code> . A change touching three fields at once emits exactly one event per changed field (the multi-field test asserts the sorted event-type list equals the three expected kinds — no more, no fewer), each naming the tool and carrying distinct baseline and current hashes. The <code>detail</code> string carries the changed values.	PASS
TOR-6	A whitespace-only difference may be labelled as such but is still reported as drift; no heuristic turns a real difference into "no drift".	<code>test_drift.py</code> : <code>whitespace_only_edit_still_drifts_but_is_labeled_as_such</code> . <code>test_fingerprint.py</code> : <code>normalize_whitespace_never_makes_a_real_content_change_disappear</code>	A whitespace-only description edit still produces a drift event (with <code>whitespace_only_change = True</code>); the event is not suppressed. <code>normalize_whitespace</code> never collapses a genuine content difference to equal.	PASS
TOR-7	Same baseline and same surface → same result on every run, independent of dict or tool-list order.	<code>test_fingerprint.py</code> : <code>fingerprint_tools_whole_server_hash_is_order_independent</code> , <code>identical_tool_hashes_identically_across_calls</code> . <code>test_drift_classification.py</code> : <code>unchanged_surfaceProduces_no_events</code> (diffs a reversed-order equal list)	Whole-server hash is identical regardless of tool order. Re-diffing an unchanged surface, including with tools listed in reverse order, yields zero events.	PASS
TOR-8	No baseline file → report "drift not evaluated", log it, never report "no drift"; the output is not usable as the §1 assurance.	<code>test_report_markers.py</code> : <code>no_baseline_session_is_not_clean</code> , <code>dashboard_and_machine_readable_flag_not_evaluated</code> , <code>evaluated_target_reports_performed</code> . <code>test_baseline_error.py</code> : <code>missing_file_returns_none</code> . Real artifact: generated <code>reports/drift-summary.json</code> .	A target whose latest proxy session logged <code>no_baseline</code> has <code>drift_status = "no_baseline"</code> , <code>is_clean = False</code> , a "DRIFT NOT EVALUATED" banner and pill on the dashboard, and <code>drift_evaluation: "not_performed"</code> in the machine-readable summary. A target with a real diff reports <code>"performed"</code> .	PASS (was partial)
TOR-9	Baseline exists but unparseable / unparseable / missing a field / unsupported version → explicit baseline-error, logged, never "no drift", no crash.	<code>test_baseline_error.py</code> (8 cases): missing→ <code>None</code> ; valid→ <code>ServerFingerprint</code> ; unparseable / non-object / missing-field / missing-version / unsupported-version→ <code>BaselineError</code> ; proxy path. Commit <code>78d5971</code> .	No case raises. Every bad-file case returns a typed <code>BaselineError</code> with a <code>reason</code> . In the proxy, <code>on_list_tools</code> writes an <code>error</code> record with <code>error_type = "baseline_error"</code> , writes no <code>tools_list_snapshot</code> and no <code>drift_event</code> , and still forwards the real result and a following tool call.	PASS (was not met)

TOR-10	Drift detection does not alter what the client receives; the real <code>tools/list</code> result is forwarded unmodified.	<code>test_forward.py</code> : <code>forward_list_tools_is_byte_identical_to_direct</code> (added 2026-08-30), <code>forward_list_tools_matches_direct_connection_exactly</code> , <code>forward_call_tool_returns_real_result_unmodified</code> . <code>test_server_side.py</code> : <code>monitor_mode_still_forwards_a_drifted_tool_call_by_default</code> . <code>test_baseline_error.py</code> asserts the real surface is forwarded even on the error path.	The forwarded <code>tools/list</code> , serialised field-for-field with the canonical dump the fingerprint uses (name / title / description / input_schema / output_schema / annotations / icons / meta), is byte-identical to a direct downstream connection — not just equal on name and description. A forwarded tool call returns content identical to a direct call. Neither a detected drift nor a skipped evaluation edits the forwarded result.	PASS
TOR-11	The prompt-injection check is outside the qualified function; its output is never represented as a qualified result and never affects the drift result.	<code>test_report_markers.py</code> : <code>scan_report_marks_injection_check_not_qualified</code> , <code>dashboard_and_detail_carry_not_qualified_marker</code> . Commit <code>a054e4a</code> .	Scan JSON carries <code>injection_check.qualification_status = "not_qualified"</code> . The scan report, dashboard, and target-detail page each render a visible "not a qualified result" marker on the injection section. The injection verdict is computed on a separate code path and never enters the drift-detection result.	PASS (was partial)

RESULT

All eleven TORs have at least one passing verification case at commit `907c8a9` . 45 cases across the seven cited test files (44 at the original Rev 2026-08-29; one added at Rev 2026-08-30, see §8); 195 in the full suite; 0 failures.

§4 Code changes this verification drove

Three of the eleven TORs did not hold against the source as MR-TQ-002 §4 recorded it. Each was closed by a change on the mcp-ratchet repository, before its verification case was executed, per the sequencing rule in MR-TQ-003 §4.

78d5971 — TOR-9 — `load_baseline` now returns a typed `BaselineError` (new dataclass in `scanner/fingerprint.py` , distinct from `None`) for a baseline file that is unreadable, unparseable, missing a required field, or an unsupported `fingerprint_schema_version` — instead of an unhandled `JSONDecodeError` / `KeyError` . `server_side.on_list_tools` gained a parallel `baseline_error` audit path alongside `no_baseline` . This is Fixes 1–3 of MR-TQ-003 §4.

a054e4a — TOR-8 and TOR-11 markers — `audit_summary` derives a per-target `drift_status` (latest session wins); `is_clean` is `False` whenever the latest session could not evaluate drift; the dashboard and detail page show a "DRIFT NOT EVALUATED" banner; a new `reports/drift-summary.json` sets `drift_evaluation` explicitly. The injection section gains a "not a qualified result" marker in all three report surfaces plus an `injection_check` field in the scan JSON. This is MR-TQ-003 §5.

907c8a9 — TOR-4 and TOR-5 coverage — the classifier cases MR-TQ-003 §9 calls for: `tool_removed` , `input_schema` change, `annotations` change, and a several-fields-at-once change (`tests/test_drift_classification.py`). No behaviour change — the classifier already handled these; the cases make the evidence explicit.

§5 TOR status after verification, against MR-TQ-002 §4

TOR	MR-TQ-002 §4 STATUS	STATUS NOW	WHAT CLOSED IT
TOR-1–7	met (tests existed)	met	re-run as verification evidence; TOR-4/5 gap cases added (<code>907c8a9</code>)
TOR-8	partial — missing baseline logged, but a report in that state could be mistaken for clean	met	a054e4a : banner, pill, <code>is_clean = False</code> , explicit <code>drift_evaluation</code> field
TOR-9	not met — unhandled exception in <code>load_baseline</code> , no schema gate	met	78d5971 + 8 verification cases
TOR-10	met	met	re-run as evidence, including on the new error path
TOR-11	partial — checks independent in code, but no report carried a "not qualified" marker	met	a054e4a : marker in scan report, dashboard, detail page; <code>injection_check</code> field in scan JSON

§6 Residual limitations

- **Independence.** Author, verifier, and notional applicant are the same person. A real program must have verification performed independently of development. Carried to MR-TQ-005.
- **Scope boundary (unchanged).** The fingerprint covers the `Tool` surface only. Resources, prompts, server-level instructions, and the content of tool-call results are not in the baseline and are covered by no TOR here. This verification does not extend the boundary — it is a real, scoped piece of future work (MR-TQ-002 §5).
- **Environment-bound.** Verified only on Windows 10.0.19045 / CPython 3.13.5 / `mcp` 2.1.1. Any change to a row in §2 invalidates the result until re-verified. The SDK moved 2.0.0 → 2.1.1 since MR-TQ-002; the `Tool` model's client-visible field set was re-checked and is unchanged, and this is recorded rather than assumed.
- **Injection check not exercised live.** The prompt-injection check needs `ANTHROPIC_API_KEY` for a real verdict; only its "key not set" path runs in this suite. It is outside the qualified function regardless (TOR-11).
- **"Latest session wins."** `drift_status` reflects the most recent proxy session. A target with an old un-evaluated session but a working latest one reads as evaluated; the historical gap is retained as a count on the detail page, not surfaced on the card. Deliberate, and documented here.

§7 Independent verification (Rev 2026-08-30)

A reviewer with no prior involvement in this repository's code or documents re-executed the whole of §3 against a clean checkout of commit `907c8a9` — the independence step MR-TQ-005 §4 calls for. Method: check the environment against §2, run `python -m pytest -q`, and for every TOR resolve each named test, run it in isolation, and read its source to confirm it asserts what the "Recorded result" column claims — adversarially, looking for prose that outruns the evidence.

CHECKED	RESULT
Environment (§2)	Matches on every row (Windows 10.0.19045, CPython 3.13.5, <code>mcp</code> 2.1.1, schema version 1).
Full suite	<code>194 passed, 0 failed, 0 skipped</code> at the reviewed commit — exactly as §2 claimed.
Counts (44 / 8)	Confirmed by <code>pytest --co</code> .
Commits <code>78d5971</code> / <code>a054e4a</code> / <code>907c8a9</code>	All real; file footprints as §4 describes.
Per-TOR assertions	Every TOR has a genuine passing case. TOR-9 rated "best-evidenced — the test asserts exactly the claim." No dishonesty: the independence limitation and the untested injection check are both disclosed, not hidden.

WHAT THE REVIEW FOUND — AND WHAT THIS REVISION DID ABOUT IT

The reviewer's verdict was **"holds with minor caveats"**: five places where the "Recorded result" prose claimed more than the cited test literally asserted, each verified correct by source inspection but not by a test. This revision closes the three that a test can close:

- **TOR-10** — "byte-matches a direct connection" was backed only by a name+description comparison. Added `test_forward_list_tools_is_byte_identical_to_direct`, which serialises both tool lists with the fingerprint's canonical dump (all eight fields) and asserts equal bytes, and strengthened the call-tool test to compare against a real direct call.
- **TOR-3 / TOR-5** — the "nothing else drifted" / "one event per field" claims are now pinned by exact `len(events)` / exact event-set assertions, not just membership checks.

Two caveats are **accepted as source-backed, not test-backed**, and stated as such here: TOR-1's eight-field canonical enumeration (it is `scanner/fingerprint.py`'s `_TOOL_FIELDS` tuple, read directly) and TOR-11's "the injection verdict never enters the drift result" (an architectural separation visible in the code, not exercised by a test). The SDK 2.0.0→2.1.1 field-set re-check (§2, §6) is likewise an author assertion with no cited artifact.

Post-revision the cited-file count is 45 and the full suite is 195; the reviewed-commit figures (44 / 194) are retained above as the record of what the independent pass actually re-executed. The reviewer noted that the §4/§5 independence condition — a *named human* reviewer on a real program — remains the governing open item; this pass was a cold re-execution and adversarial read, which substantively addresses "independent of development" but is not a substitute for program-level QA sign-off.

§8 Carried forward to MR-TQ-005

- The verdict: whether the drift-detection function is qualified at TQL-5 for the credited use in MR-TQ-001 §3, on the evidence in §3.
- The independence limitation (§1, §6, §7): a cold re-execution and adversarial review was done at Rev 2026-08-30 (§7); a named human reviewer on a real program is still required.
- The three evidence-precision caveats the review left open as source-backed (TOR-1 field list, TOR-11 code separation, the SDK re-check), restated in MR-TQ-005 as accepted limitations.
- The scope boundary (§6) as an explicit statement of what a qualified "no drift" result does *not* cover.
- Configuration management actions: tag 907c8a9 (or its successor, if the set is revised) as the qualified commit; record the SHA-256 of the credited assistant's baseline file under program CM (MR-TQ-003 §6).